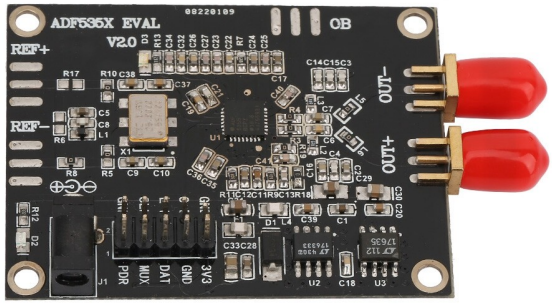
Raspberry Pi + ADF5355

Ku-band duration-coded ladder: 10.7 to 12.7 GHz in an 18-second closed-loop bench pattern

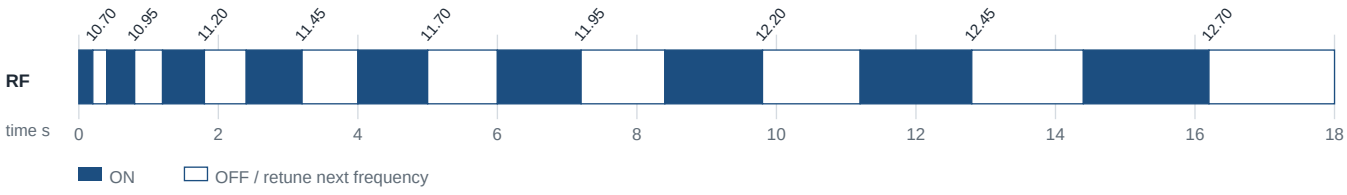
Bench-test boundary: this guide assumes OB/RFOUTB is coupled only into a shielded, heavily attenuated or otherwise controlled test path. The 10.7-12.7 GHz range overlaps satellite downlink spectrum. Do not connect an antenna and radiate the ladder into free space.



Your board: use **OB** (ADF5355 RFOUTB) for 10.7-12.7 GHz. Disable OUT+ and OUT- (RFOUTA) in software; when disabled, they can remain open.

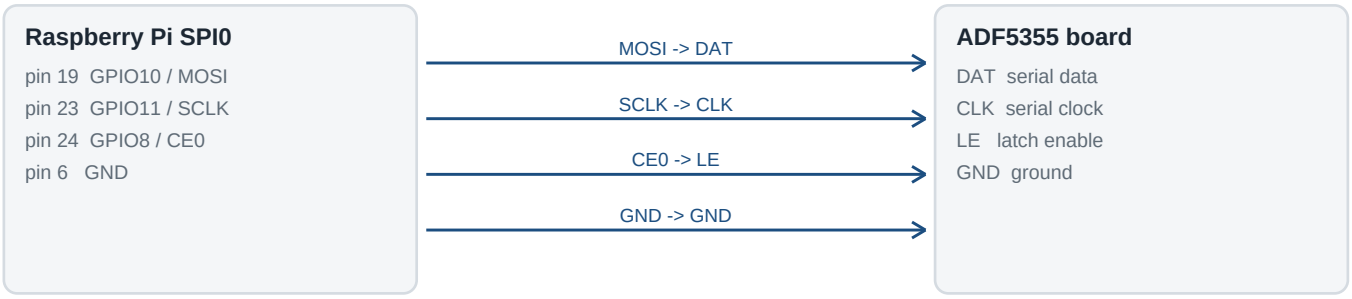
The default ladder

Nine CW carriers are stepped from 10.700 to 12.700 GHz in 250 MHz increments. Rung *n* is ON for *n x 0.200 s*, then OFF for the same time. The sum of all ON and OFF windows is exactly 18.000 s. A 1.4 s burst, for example, identifies rung 7 and therefore 12.200 GHz.



1. Connect the Raspberry Pi

Use the Pi hardware SPI0 interface. The ADF5355 accepts 32-bit register words; data is shifted MSB-first and latched on the rising edge of LE. Wiring Pi CE0 to the board LE pad makes each 4-byte spidev transaction generate that latch edge.



MISO is not used. Leave MUX/PDR unconnected for the baseline setup.

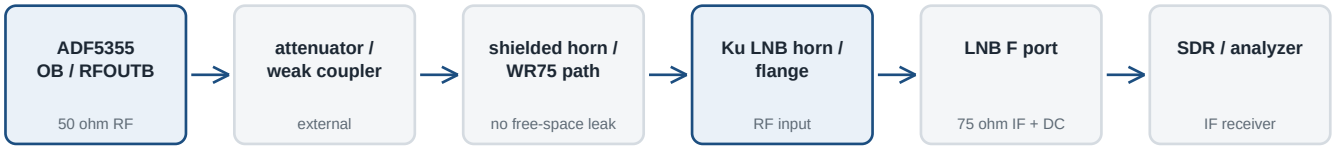
Pi signal	Physical pin	Board signal	Purpose
GPIO10 / MOSI	19	**DAT**	Serial data into ADF5355
GPIO11 / SCLK	23	**CLK**	Serial clock
GPIO8 / CE0	24	**LE**	Latch enable; rises after each 32-bit transfer
GND	6 (or any GND)	**GND**	Common digital ground

Clone-board warning: the photo clearly shows DAT/GND/3V3 on one row, but the CLK/LE locations are partly obscured. Connect by *signal name*, not by guessed header position. Verify CLK and LE from the silkscreen, seller schematic, or continuity to the ADF5355 before powering.

Power: power the synthesizer through its intended DC input at the voltage specified for your exact module. Do not use the Pi 3V3 pin to power the synthesizer, and do not tie Pi 3V3 to the board 3V3 rail when the board is separately powered. MISO is not required. Leave MUX and PDR in the module's normal/default state for this baseline test.

OUT+ / OUT-: the supplied script sets RFOUTA enable = 0. With RFOUTA disabled, leave OUT+ and OUT- open. The only active RF connector is OB/RFOUTB.

2. Build the closed RF path



Do not put 10-13 GHz RF into the LNB F connector. The horn/flange is the RF input.

RFOUTB is the doubled-VCO output and covers 6.8-13.6 GHz, which is why OB is used for this test. The script mutes/unmutes RFOUTB by changing Register 6 DB10. Unlike RFOUTA, ADF5355 RFOUTB does not provide the same programmable output-power field in the ADI driver, so control signal level with **external attenuation/coupling**.

Start with very weak coupling. An LNB is a high-gain receive front end. Use a shielded box, WR75/waveguide attenuator, directional coupling, or substantial separation inside a metal enclosure. Increase coupling only until the IF tone is easy to see.

For a conventional universal Ku LNB, the F connector is a 75-ohm **IF output plus DC/control input**. Feed the LNB from a proper bias tee / satellite receiver / LNB controller; do not power it from a Pi GPIO. A common universal arrangement uses a 9.75 GHz low-band LO and 10.6 GHz high-band LO, with a 22 kHz control tone selecting high band. Verify your specific LNB datasheet.

Rung	RF GHz	Width ON/OFF	Typical LNB mode	Expected IF GHz
1	10.700	0.200 s	low, LO 9.75	0.950
2	10.950	0.400 s	low, LO 9.75	1.200
3	11.200	0.600 s	low, LO 9.75	1.450
4	11.450	0.800 s	low, LO 9.75	1.700
5	11.700	1.000 s	band boundary	1.950 low / 1.100 high
6	11.950	1.200 s	high, LO 10.60	1.350
7	12.200	1.400 s	high, LO 10.60	1.600
8	12.450	1.600 s	high, LO 10.60	1.850
9	12.700	1.800 s	high, LO 10.60	2.100

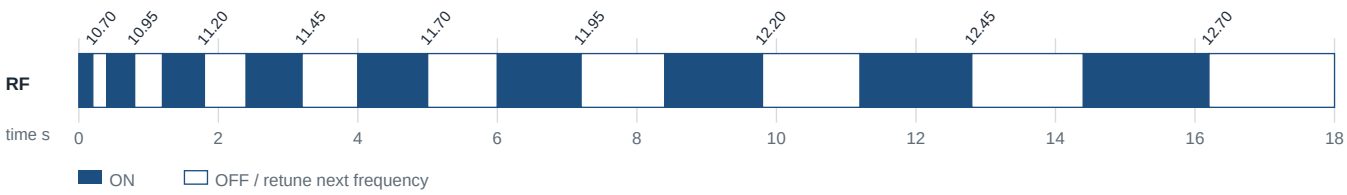
To observe all nine rungs in one pass with a switched universal LNB, the receive-side controller must be in the correct low/high band at the 11.7 GHz boundary. If your setup cannot switch during the run, capture low and high portions separately or use a wideband/appropriate LNB architecture.

3. The 18-second self-identifying ladder

The duration is part of the code. For N=9 rungs and total time T=18 s, choose a base unit:

u = T / [N(N+1)] = 18 / (9 x 10) = 0.200 seconds

Then rung n uses ON = n*u and OFF = n*u. The equal OFF window makes the cadence recognizable even if a recording starts in the middle of the pattern. Frequency is a second identifier: f(n) = 10.700 + 0.250(n-1) GHz.



n	RF GHz	ON	OFF	Starts	ON ends	Rung ends
1	10.700	0.200s	0.200s	0.0s	0.2s	0.4s
2	10.950	0.400s	0.400s	0.4s	0.8s	1.2s
3	11.200	0.600s	0.600s	1.2s	1.8s	2.4s
4	11.450	0.800s	0.800s	2.4s	3.2s	4.0s
5	11.700	1.000s	1.000s	4.0s	5.0s	6.0s
6	11.950	1.200s	1.200s	6.0s	7.2s	8.4s
7	12.200	1.400s	1.400s	8.4s	9.8s	11.2s
8	12.450	1.600s	1.600s	11.2s	12.8s	14.4s
9	12.700	1.800s	1.800s	14.4s	16.2s	18.0s

Decoding example: if the IF carrier is present for about 1.4 s and then absent for about 1.4 s, n = 1.4/0.2 = 7, so the corresponding RF rung is 12.200 GHz. If a fragment contains only the OFF interval, the same duration code still applies.

The Python program retunes the *next* frequency during the current OFF window. That keeps the coded first-ON to final-OFF interval at 18 seconds rather than adding PLL programming time between rungs. Linux/Python is not hard real time, so expect small scheduling jitter; for laboratory timing measurements below the millisecond level, move the timing engine to a microcontroller/FPGA.

4. Prepare Raspberry Pi OS

Use a current Raspberry Pi OS installation. SPI0 is disabled by default on many images; enable it once, then install Python spidev.

```
sudo raspi-config
# Interface Options -> SPI -> Enable

sudo apt update
sudo apt install -y python3-spidev

ls -l /dev/spidev0.0
```

Pi SPI0 pin mapping used by the script: MOSI=GPIO10/pin 19, SCLK=GPIO11/pin 23, CE0=GPIO8/pin 24. The script does not use MISO.

Copy the companion file **adf5355_ladder.py** to the Pi, for example into `~/adf5355/`, and make it executable:

```
mkdir -p ~/adf5355
cd ~/adf5355
# copy adf5355_ladder.py here
chmod +x adf5355_ladder.py
```

The most important input is the board reference oscillator frequency. This clone family appears with several X1 values. The program deliberately has **no default** for this parameter.

> **Read X1 before transmitting. If X1 is marked 100.000 MHz, use --ref-mhz 100. If it is 122.880 MHz, use 122.88; if 125.000 MHz, use 125; and so on. A wrong reference value produces the wrong RF frequencies even if the PLL locks.**

First run: calculate only, no RF.

```
cd ~/adf5355
python3 adf5355_ladder.py --ref-mhz 100 --dry-run
```

Replace 100 with the actual X1 frequency. Dry-run prints the nine-rung schedule, the calculated PFD and first register image without opening SPI or enabling RF.

Second run: hardware connected, but still no RF.

```
python3 adf5355_ladder.py --ref-mhz 100
```

This opens the synthesizer only if needed, but without `--enable-rf` the program exits without generating the ladder. This is an intentional safety interlock.

5. Run one 18-second ladder

Only after the OB path is shielded/attenuated, the LNB is correctly powered, and the reference value is verified:

```
python3 adf5355_ladder.py --ref-mhz 100 --enable-rf
```

If your user account cannot open /dev/spidev0.0, fix SPI permissions or, for a quick bench check, run the same command with sudo.

The default behavior is intentionally conservative:

- **One ladder only.** Default --loops 1; no infinite mode.
- **RFOUTA is disabled.** OUT+ and OUT- remain inactive/open.
- **RFOUTB starts muted.** The first frequency is programmed and allowed to settle before t=0.
- **Retuning happens while muted.** The next carrier is programmed during the OFF half of the current rung.
- **Ctrl-C mutes RFOUTB.** The exception/finally path attempts to return Register 6 DB10 to the disabled state.
- **Reference divider is automatic.** The script chooses R so the phase-frequency detector stays at or below 75 MHz, following the ADI driver approach.

Useful options

```
# Same ladder, repeated three times (each interval still 18 s)
python3 adf5355_ladder.py --ref-mhz 100 --loops 3 --enable-rf

# Change the number of rungs while preserving the requested total time
python3 adf5355_ladder.py --ref-mhz 100 --steps 8 --total-s 18 --dry-run

# Override the starting charge-pump setting if your board documentation requires it
python3 adf5355_ladder.py --ref-mhz 100 --cp-ua 900 --dry-run
```

The supplied 900 uA charge-pump default follows the common ADI evaluation/driver starting point. A clone may use a different loop filter. If the PLL does not lock reliably, verify the exact module schematic/reference/loop-filter values rather than blindly changing registers.

6. What the script is doing

For RFOUTB, the requested Ku-band carrier is twice the ADF5355 fundamental VCO frequency. Thus 10.700 GHz programs a 5.350 GHz VCO, and 12.700 GHz programs a 6.350 GHz VCO. Both are inside the 3.4-6.8 GHz VCO range.

Stage	Operation
SPI write	One 32-bit register word is sent as four bytes, MSB first. CE0 is used as LE and rises at the end of the transaction.
PLL math	The program computes INT, FRAC1, FRAC2 and MOD2 from the requested VCO and actual PFD frequency, following ADI no-OS formulas.
Initialization	Registers 12 down to 1 are written, the required ADC-clock delay is observed, then Register 0 is written with autocal enabled.
Frequency change	R10, R6, R4(counter reset), R2, R1, R0(no autocal), normal R4, delay, then R0(autocal) - matching the ADI update sequence.
ON/OFF keying	Register 6 DB10 is cleared to enable ADF5355 RFOUTB and set to disable it. The A output enable bit remains zero.

Troubleshooting

- **No signal anywhere:** verify the board power rails first, then run dry-run, then confirm SPI activity on DAT/CLK/LE with a scope or logic analyzer.
- **Carrier exists but frequency is wrong:** the most likely first check is X1/reference frequency. Do not infer it from this photo.
- **5-6 GHz appears on OUT+/- but not 10-13 GHz on OB:** confirm RFOUTB is physically connected/populated and that the board is actually an ADF5355-compatible variant. The script intentionally disables RFOUTA.
- **PLL hops but receiver misses some steps:** check LNB low/high-band selection around 11.7 GHz and confirm its IF passband. A universal LNB normally cannot show the full 10.7-12.7 range in one fixed band state.
- **LNB/SDR overload:** reduce coupling / add attenuation. The test should need only a tiny fraction of the synthesizer output at the LNB input.
- **Timing is a few milliseconds off:** Linux scheduling is not deterministic. The duration code is intended for human/receiver identification, not precision metrology.

7. Pre-flight checklist and references

- [ ] OB/RFOUTB is the only RF port connected to the test path.
- [ ] OUT+ and OUT- are left open because RFOUTA is disabled in software.
- [ ] The RF path is shielded / attenuated and is not connected to a free-space antenna.
- [ ] Pi MOSI -> DAT, SCLK -> CLK, CE0 -> LE, GND -> GND have been verified.
- [ ] The synth board is powered from its specified supply, not from Pi 3V3.
- [ ] X1/reference oscillator value has been read and entered with --ref-mhz.
- [ ] Dry-run output looks sensible before --enable-rf is used.
- [ ] LNB is powered through a correct bias tee/controller; its F connector is treated as IF/DC, not 11 GHz RF input.
- [ ] Receiver is prepared for the low/high LNB band change around 11.7 GHz if using a universal LNB.
- [ ] A Ctrl-C test has been done with the RF path safely shielded to confirm the script mutes on exit.

Primary technical references

Analog Devices ADF5355 product page / datasheet
RFOUTB 6.8-13.6 GHz; RFOUTA to 6.8 GHz; register definitions and timing.
https://www.analog.com/en/products/adf5355.html

Analog Devices no-OS ADF5355 driver (C)
Register write order, PLL calculation, PFD divider, ADC delay and update sequence.
https://github.com/analogdevicesinc/no-OS/blob/main/drivers/frequency/adf5355/adf5355.c

Analog Devices no-OS ADF5355 header
Bit fields, defaults, RFOUTB enable semantics and frequency limits.
https://github.com/analogdevicesinc/no-OS/blob/main/drivers/frequency/adf5355/adf5355.h

Raspberry Pi official SPI documentation
SPI0 GPIO mapping and enabling the Linux SPI driver.
https://www.raspberrypi.com/documentation/computers/raspberry-pi.html

Inverto universal Ku LNB technical data
Example primary manufacturer data for 10.7-12.75 GHz, 9.75/10.6 GHz LOs and 22 kHz band switching.
https://inverto.tv/lnb/546/quad-universal-40mm-pll-lnb

Companion file: adf5355_ladder.py is supplied alongside this PDF. Keep the PDF and script together so the run commands, safety assumptions and implementation stay synchronized.

Revision note: written for the ADF535X EVAL V2.0-style board shown in the supplied photograph. Clone modules vary. The exact oscillator, DC input requirement and header order must be confirmed on the user's physical board.